

Computability is Bollocks

M-reducibility:

For two TMs A and B over the same alphabet Σ^* , A is m -reducible to B if there exists some TM F such that

$$A = \{u \in \Sigma^* \mid F(u) \in B\}$$

→ i.e. there exists some F such that applying F to each input instance of B causes B to behave exactly like A .

This means that checking some input is accepted by A is no harder than checking some input is accepted by B .

Note: F must always terminate on all $u \in \Sigma^*$

" A is m -reducible to B " is written $A \leq_m B$.

If $A \leq_m B$, then if B is decidable/recognisable, so is A .

If $A \leq_m B$ and $B \leq_m C$, then $A \leq_m C$

If $A \leq_m B$ and $B \leq_m C$, then $A \equiv B$ - A and B are equally difficult.

M-completeness:

A language A is m -complete if it is recognisable and for every recognisable language

$$B, B \leq_m A.$$

⇒ m -complete languages are at least as hard as any Turing-recognisable language.

If A is m -complete and $A \leq_m B$, then B is also m -complete.

The Halting Language H is m -complete.

Proof: We know that H is recognisable.

Take any arbitrary recognisable language A .

Since it is recognisable, there exists some TM, M_A , that recognises A - i.e. it halts and accepts on all inputs in A . Construct this machine such that it does not halt on inputs not in A - this is possible for any TM by modifying the transition function to an infinite loop where the reject state was and make the reject state unreachable.

For H to be m -complete, need $M_A \leq_m H$.

Consider H with input $\langle M_A \rangle w$. Since M_A only halts on w iff $w \in A$,

$$w \in A \iff \langle M_A \rangle w \in H$$

$\Rightarrow$ Any arbitrary recognizable language can be m -reduced to H

Since H is recognizable, H is m -complete.

Since we know H is m -complete, we can prove some other language B is m -complete by showing that B is recognizable and by m -reducing H to B , i.e. proving $H \leq_m B$.

$\rightarrow$ We generally only need to show a transformation from H to a specific, constructed instance of B based on a general instance of H .

Example: prove that the problem of determining whether two Turing Machines A and B decide different languages is m -complete.

Determining the problem is recognizable:

Go through all possible inputs in some order, and run both A and B on each. Accept (and halt) if for any input they disagree — we know that A and B decide different languages.

Reduction from H :

Consider an instance of H , $\langle M \rangle w$ (such that H accepts iff M halts on w).

We need to construct machines A and B such that A and B decide different languages, i.e. $L(A) \neq L(B)$, iff M halts on w .

Construct A such that $L(A) = \emptyset$ — i.e. A rejects every input.

Construct B to decide the step-counter predicate for M and w , such that for some k B accepts iff M terminates on w in $\leq k$ steps.

Now:

If M halts on w in some finite k steps, A accepts — i.e. $L(A) \neq \emptyset = L(B)$

If M does not halt on w , there does not exist a finite k such that M terminates on w in $\leq k$ steps. Therefore A rejects — $L(A) = \emptyset = L(B)$

$\Rightarrow A$ and B decide different languages iff M halts on w

$\Rightarrow$ The problem is m -complete.

+-reduction:

An oracle for a language A is a black-box that takes a word w as input and instantly and correctly outputs whether $w \in A$.

An oracle Turing Machine M^A is a TM with the ability to make calls to the oracle for A .

A language A is +-reducible to a language B , denoted $A \leq_t B$, if A is decidable by some oracle TM M^B .

If $A \leq_t B$ and B is Turing-decidable, then A is also Turing-decidable.

If $A \leq_m B$, then $A \leq_t B$ but the opposite is not always true.

Computable and Partially Computable Functions

A **total function** $f: \Sigma^* \rightarrow \Sigma^*$ is a function that returns a defined result for all inputs $\in \Sigma^*$. Such a function is **computable** if there exists a TM F such that for any input $x \in \Sigma^*$, F outputs $f(x)$.

A **partial function** $g: \Sigma^* \rightarrow \Sigma^*$ is a function whose domain is a subset of inputs $\text{dom}(g) \subset \Sigma^*$, and is not defined for inputs not in $\text{dom}(g)$. Such a function is **partially computable** if there exists a TM G such that for any input $x \in \text{dom}(g)$, G outputs $g(x)$ and for all inputs $x \notin \text{dom}(g)$, G doesn't terminate.

A language is **Turing-Recognisable** iff it is:

- the domain of a **partially computable function** $\rightarrow$ terminates on all positive instances
- the range of a **computable function**
- the range of a **partially computable function**

Primitive Recursion

The **initial functions** are:

- Successor $\rightarrow S(x) = x+1$
- Zero $\rightarrow \eta(x) = 0$
- projection $\rightarrow \pi_i^n(x_1, \dots, x_n) = x_i \quad \forall n \in \mathbb{N}, 1 \leq i \leq n$

Let f be a function of k variables, let $g_1, \dots, g_k$ be functions of n variables.

The function h of n variables is obtained from f and $g_1, \dots, g_k$ by composition if:

$$h(x_1, \dots, x_n) = f(g_1(x_1, \dots, x_n), \dots, g_k(x_1, \dots, x_n))$$

Let f and g be **total functions** of n variables and $n+2$ variables respectively.

The function h of $n+1$ variables is obtained from f and g by primitive recursion if:

$$h(x_1, \dots, x_n, 0) = f(x_1, \dots, x_n) \rightarrow \text{base case}$$

$$h(x_1, \dots, x_n, t+1) = g(t, h(x_1, \dots, x_n, t), x_1, \dots, x_n) \rightarrow \text{recursion.}$$

A function is called **primitive recursive** if it can be obtained from the **initial functions** by a finite number of **composition** and **primitive recursion** operations.

Primitive recursive functions include addition, multiplication, exponentiation, ...

All primitive recursive functions are **computable**.

Let f be a function of $n+1$ variables. The function h of n variables is obtained from f by minimisation if:

$$h(x_1, \dots, x_n) = \begin{cases} \text{the smallest } y \text{ such that } f(x_1, \dots, x_n, y) = 0 \\ \text{undefined if no such value exists.} \end{cases}$$

A function is called **partially recursive** if it can be obtained from the **initial functions** by a finite number of **composition**, **primitive recursion** and **minimisation** operations.

All partially recursive functions are **partially computable**.

Gödel's Incompleteness

The **Gödel Number** of a sequence of numbers $x_1, \dots, x_n$ is defined as $p_1^{x_1} \dots p_{n-1}^{x_{n-1}} p_n^{x_n+1}$, where p_i is the i 'th prime number. The process of Gödel numbering is primitive recursive.

$\Rightarrow$ We add 1 to the power of the last prime so that each Gödel number is unique

Otherwise: $67 \rightarrow 2^6 \cdot 3^7$
 $670 \rightarrow 2^6 \cdot 3^7 \cdot 5^0 = 2^6 \cdot 3^7$
 $6700 \rightarrow 2^6 \cdot 3^7 \cdot 5^0 \cdot 7^0 = 2^6 \cdot 3^7$
 Trailing zeros in the sequence do not change the Gödel number $\Rightarrow$ bad.

With the +1: $67 \rightarrow 2^6 \cdot 3^8$
 $670 \rightarrow 2^6 \cdot 3^7 \cdot 5^1$
 $6700 \rightarrow 2^6 \cdot 3^7 \cdot 5^0 \cdot 7^1$ } all unique - we can decode it and get the original sequence.

We can represent TMs and input strings as Gödel numbers, denoted $[x]$.
 Gödel numbering allows Maths to reason about itself.

Gödel's Incompleteness Theorem: there are statements which are True but are not provable.

Proof: A formula ϕ can be encoded by its Gödel number $[\phi]$.

A Proof π can be encoded by its Gödel number $[\pi]$

Let $\text{Proof}([\pi], [\phi])$ state that π is a proof of ϕ - this is primitive recursive.

Assume that the system is consistent and that all true statements are provable, i.e.

$$\exists p \text{ Proof}([p], [f])$$

for any true formula f .

Let f be the Co-Halting problem. For every positive instance M, w of the Co-Halting problem:

$$\exists p \text{ Proof}([p], [M \text{ does not terminate on } w])$$

positive instance $\Rightarrow$ Co-H is True $\Rightarrow$ by our assumption, all true statements are provable

Since, by our assumption, a proof exists, we can find it by brute force. Therefore, the

Co-Halting problem is **Semi-decidable** $\Rightarrow$ the Halting Problem is **decidable**.

This is a contradiction, therefore our assumption that all true statements are provable is false.

Proof (self-referential sentence):

Assuming True $\Leftrightarrow$ Provable:

Let $\phi(x)$ be some formula with one free variable.

Let $P(n, [\phi])$ be a predicate, meaning " n does not encode a proof of $\phi([\phi])$ "

Let $\psi(y)$ be $\forall x P(x, y) \rightarrow$ "does every x not encode a proof of $y([\psi])$?"

$\psi([\psi]) \rightarrow$ "does every x not encode a proof of $\psi([\psi])$?"

$\psi([\psi]) = \text{True} \Rightarrow$ every x does not encode a proof of $\psi([\psi]) \Rightarrow \psi([\psi]) = \text{False}$
 assuming everything True is provable

$\psi([\psi]) = \text{False} \Rightarrow$ Some x encodes a proof $\Rightarrow \psi([\psi]) = \text{True}$
assuming everything provable is true

This is a contradiction $\rightarrow$ assumption is false.